

DATA ENGINEERING LEARNING PATH

PySpark by Practice → Big Data Engineering Roadmap

A hands-on tutorial built around your own customer dimension dataset (18,484 rows) — from Spark fundamentals through the exact skills big data engineering roles test for: partitioning, window functions, performance tuning, and lakehouse formats.

Practice dataset: gold_dim_customers.csv

PySpark 4.x / Python 3.12

Every code block below is verified and executable

Spark Core

DataFrame API

Window Functions

Spark SQL

Parquet / Delta Lake

Partitioning & Joins

Airflow

Structured Streaming

1 Where PySpark Fits in Big Data Engineering

Understanding the stack before diving into syntax

You already have pandas, T-SQL, and PySpark exposure from your EDA and medallion architecture work. The gap between "I can write PySpark" and "I can be a data engineer" is mostly about **why Spark exists** and **how it behaves at scale** — not new syntax.

CORE IDEA

Pandas loads all data into one machine's memory. Spark splits data into **partitions** spread across a cluster of machines (or cores, locally), and runs the same transformation on each partition in parallel. That single idea explains almost every Spark quirk you'll hit: lazy evaluation, shuffles, the driver/executor split, and why "it works fine on my laptop but breaks on 10TB" is a real category of bug.

The typical big data engineering stack

LAYER	WHAT IT DOES	COMMON TOOLS
Ingestion	Pull data from source systems into the lake	Kafka, Debezium (CDC), Fivetran, Airbyte
Storage (lakehouse)	Cheap, scalable columnar storage	Parquet, Delta Lake, Apache Iceberg, S3 / ADLS
Processing	Transform raw → clean → aggregated (medallion: bronze/silver/gold)	PySpark , Spark SQL, dbt
Orchestration	Schedule and chain pipeline steps	Airflow, Dagster, Databricks Workflows
Serving	Expose results to BI / apps	Warehouses (Snowflake, BigQuery), APIs

You've already built the medallion architecture in SQL. PySpark is what runs those same bronze → silver → gold transformations when the data is too large, or too fast-arriving, for a single SQL Server instance to handle comfortably.

WHY THIS MATTERS FOR YOU SPECIFICALLY

Every DataFrame operation below has a direct T-SQL equivalent you already know. Treat this less like "learning a new language" and more like "learning a new execution engine for concepts you've already internalized."

2 The Learning Roadmap

Four stages, ordered by leverage — do them in order

STAGE 1 Spark Fundamentals ~1 week

SparkSession, RDD vs DataFrame, lazy evaluation, transformations vs actions, reading/writing CSV & Parquet, basic select/filter/groupBy. **You are ~80% here already.**

STAGE 2 Core DataFrame Mastery ~2 weeks

Window functions (rank, row_number, lead/lag, running totals), joins (inner/left/anti/semi), complex types (structs, arrays), Spark SQL, UDFs vs native functions, null handling patterns.

STAGE 3 Performance & Internals ~2-3 weeks

Catalyst optimizer & physical plans, shuffles and why they're expensive, broadcast joins, partitioning strategy (partitionBy, repartition vs coalesce), caching/persistence, skew handling.

STAGE 4 Production Data Engineering ongoing

Delta Lake / Iceberg (ACID tables, time travel, schema evolution), Structured Streaming, orchestration with Airflow, testing pipelines, deploying to Databricks/EMR/Synapse.

COMMON TRAP

Don't skip to Stage 4 tools because they sound impressive on a CV. Interviewers for data engineering roles spend most of their time probing Stage 2–3: "walk me through why this join is slow" is a far more common interview question than "have you used Delta Lake."

3 Your Practice Project

Built and verified against `gold_dim_customers.csv` — run it yourself

The accompanying `pyspark_practice.py` script runs end-to-end against your 18,484-row customer dimension file and covers 10 sections, each mapping to a roadmap stage above. It is not pseudocode — every line executes.

3.1 — Schema: inferred vs. explicit

```
# Explicit schema = one file read, predictable types, fails fast on bad rows.
# This is what production ingestion jobs use – inferSchema=True is for exploration only.
schema = StructType([
    StructField("customer_key", IntegerType(), True),
    StructField("birthdate", DateType(), True),
    # ...
])
df = spark.read.csv("gold_dim_customers.csv", header=True, schema=schema)
```

3.2 — A real dirty-data gotcha your dataset actually contains

FOUND IN YOUR DATA

337 rows have a genuinely NULL `country` — but a *separate* set of rows store the literal text `"n/a"` as a string. `isNull()` and `coalesce()` silently miss the second case because `"n/a"` is a perfectly valid string as far as Spark is concerned. Always eyeball `groupBy(col).count()` before trusting a null report.

```
# Fixes BOTH the true-null and the "n/a" sentinel-string cases:
df_clean = (
    df.replace("n/a", None, subset=["country", "gender"])
    .withColumn("country", F.coalesce(F.col("country"), F.lit("Unknown")))
)
```

3.3 — Window functions (your highest-leverage next skill)

```
country_window = Window.partitionBy("country").orderBy(F.desc("age"))

df.withColumn("age_rank_in_country", F.row_number().over(country_window)) \
    .filter(F.col("age_rank_in_country") <= 3)
```

This is the direct PySpark equivalent of the T-SQL `RANK() OVER (PARTITION BY ... ORDER BY ...)` pattern you already learned. Same concept, same mental model — different API.

3.4 — Broadcast joins

```
# Small lookup table (country -> region) gets copied to every executor
# instead of triggering an expensive shuffle of the large table.
df.join(F.broadcast(country_region), on="country", how="left")
```

3.5 — Partitioned Parquet write (how lakehouses store data)

```
df.write.mode("overwrite").partitionBy("country").parquet("output/customers_by_country")
# Produces: output/customers_by_country/country=Australia/*.parquet, etc.
# This is EXACTLY how a bronze/silver/gold lake is physically laid out on disk.
```

HOW TO USE THE SCRIPT

Run `python3 pyspark_practice.py` section by section. After each one, close the file and try to reproduce the output from memory before looking. That retrieval effort is what actually builds the skill — reading code you didn't struggle with barely sticks.

4 Translation Cheat Sheet

Mapping what you already know onto PySpark syntax

TASK	PANDAS	T-SQL	PYSPARK
Filter rows	<code>df[df.age > 30]</code>	<code>WHERE age > 30</code>	<code>df.filter(F.col("age") > 30)</code>
Select columns	<code>df[['a', 'b']]</code>	<code>SELECT a, b</code>	<code>df.select("a", "b")</code>
New column	<code>df['x'] = df.a + df.b</code>	<code>a + b AS x</code>	<code>df.withColumn("x", F.col("a")+F.col("b"))</code>
Group + aggregate	<code>df.groupby('c').agg(...)</code>	<code>GROUP BY c</code>	<code>df.groupBy("c").agg(...)</code>
Rank within group	<code>df.groupby('c').rank()</code>	<code>RANK() OVER (PARTITION BY c ...)</code>	<code>F.rank().over(Window.partitionBy("c")...)</code>
Join	<code>df.merge(df2, on='k')</code>	<code>JOIN ... ON k</code>	<code>df.join(df2, on="k", how="inner")</code>
Drop nulls	<code>df.dropna()</code>	<code>WHERE col IS NOT NULL</code>	<code>df.na.drop()</code>
Sort	<code>df.sort_values('a')</code>	<code>ORDER BY a</code>	<code>df.orderBy("a")</code>

KEY MENTAL SHIFT FROM PANDAS

pandas executes every line immediately. PySpark builds a plan (a DAG) and only executes when you call an *action* —

`.count()` , `.show()` , `.collect()` , or `.write(...)` . Nothing "happens" at `.filter()` or `.withColumn()` time. This is the single most common source of confusion coming from pandas.

5 Performance Tuning — What Interviews Actually Test

Stage 3 material, condensed

5.1 — Shuffles: the thing that makes Spark jobs slow

A **shuffle** happens whenever data with the same key needs to end up on the same partition — think `groupBy`, non-broadcast `join`, or `repartition`. It means writing data to disk and moving it across the network between executors. Minimizing unnecessary shuffles is most of what "Spark performance tuning" means in practice.

5.2 — Partitioning decisions

OPERATION	WHAT IT DOES	WHEN TO USE
<code>repartition(n)</code>	Full shuffle, evens out partition sizes	Before a big write, or after a heavy filter that left partitions lopsided
<code>coalesce(n)</code>	Merges partitions without a full shuffle	Reducing partition count cheaply (e.g. before writing small output)
<code>partitionBy(col)</code> on write	Physically splits output files by column value	Columns you'll filter on later (dates, country, region)

5.3 — Quick reference: symptoms → fixes

SYMPTOM	LIKELY CAUSE	FIX
One task takes forever, others finish fast	Data skew — one key has way more rows	Salt the key, or isolate & handle it separately
Job is slow even on small data	Too many small partitions (task overhead)	<code>coalesce()</code> before writing
Join is slow	Both sides are being shuffled	<code>F.broadcast()</code> the smaller side if it fits in memory
Same DataFrame recomputed repeatedly	No caching, lazy plan reruns each time	<code>df.cache()</code> / <code>.persist()</code> if reused 2+ times

DON'T OVER-CACHE

Caching costs memory. Only cache a DataFrame if it's reused multiple times downstream — caching something used once is pure overhead.

6 Next Steps & Practice Ideas

Once the script above feels comfortable

Extend this same dataset

- ✓ Add a synthetic `orders` table and practice fact/dimension joins
- ✓ Build a full bronze → silver → gold pipeline as three chained scripts
- ✓ Simulate late-arriving data and handle it with watermarking
- ✓ Introduce duplicate rows and dedupe with window functions
- ✓ Convert the Parquet output to Delta Lake and try time travel queries

Concepts to research next

Adaptive Query Execution (AQE)

Delta Lake MERGE

Structured Streaming

Airflow DAGs

Z-ordering

Slowly Changing Dimensions

dbt + Spark

Databricks Unity Catalog

Free resources

- ✓ Official Spark docs — Structured API guide
- ✓ Databricks Academy (free tier courses)
- ✓ "Learning Spark, 2nd Edition" (O'Reilly, free PDF from Databricks)

Built as a practice companion around your own `gold_dim_customers.csv` dataset — every code snippet in this document is drawn from the verified, runnable `pyspark_practice.py` script delivered alongside it.